BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

INTRODUCTION TO WEB PROGRAMMING

Unit 13

Understanding APIs in Web Development

Table of Contents

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	3 - 4
	1.1 Objectives	-	-	
2	Introduction To Api	-	-	5-11
3	Understanding Rest APIs	-	-	11-14
4	Fetching Data From Apis Using Javascript	-	-	14-20
5	Summary	-	-	21
6	Self-Assessment Questions	-	-	22
7	Terminal Questions	-	-	23
8	Answers To Self Assessment Questions	-	-	23-24
9	Answers To Terminal Questions	-	-	24

1. INTRODUCTION

In today's interconnected digital landscape, the exchange of information between different software applications and services is at the core of modern web development. At the heart of this data exchange lies Application Programming Interfaces (APIs), which serve as the bridge enabling seamless communication between diverse systems. But what exactly is an API? In this opening chapter, we embark on a journey to demystify the concept of APIs, breaking down the acronym and exploring their fundamental purpose. We'll delve into the world of web APIs, with a specific focus on Representational State Transfer (REST) APIs, a widely adopted architectural style for designing networked applications. This chapter will lay the foundation for understanding how APIs work, why they're crucial in web development, and how you can harness their power to retrieve data and enhance the functionality of your web applications using JavaScript.

With the basics of APIs and REST under our belt, we'll then turn our attention to the practical side of things. Building upon our newfound knowledge, we will explore how to use JavaScript, one of the most popular programming languages for web development, to interact with APIs and fetch data from remote servers. You'll learn about the different methods, libraries, and best practices for making HTTP requests to APIs, handling responses, and integrating this data into your web applications. By the end of this chapter, you'll have a solid understanding of the essential concepts, tools, and techniques required to fetch and manipulate data from APIs using JavaScript, paving the way for the exciting journey ahead as we dive deeper into the world of web development and data integration.

1.1. Objectives:

At studying this unit, you should be able to:

- *Define what an API is and explain the fundamental principles of REST APIs, including key architectural concepts and HTTP methods commonly used in API interactions.*
- *Demonstrate an understanding of the role of APIs in web development, including the benefits of using REST as an architectural style, and describe how APIs facilitate communication between different software systems.*

- *Apply JavaScript programming skills to initiate HTTP requests to RESTful APIs, interpret API documentation, and formulate appropriate requests to fetch data from remote servers.*
- *Analyze API responses and effectively handle data, including parsing JSON data, error handling, and understanding status codes, while identifying potential issues in the retrieval process.*
- *Design and create web applications that utilize data fetched from REST APIs, integrating this data into user interfaces to enhance user experiences and demonstrate an ability to troubleshoot and optimize API interactions in real-world scenarios.*

2. INTRODUCTION TO API

An Application Programming Interface (API) serves as an intermediary that enables different software applications to communicate with each other. It defines a set of rules and protocols, allowing one piece of software to request and interact with specific functions or data from another. APIs have become an integral part of modern software development, acting as bridges that facilitate the exchange of information and functionality between diverse systems.

Let's delve into the applications of APIs in more detail:

1. Social Media Integration:

- Facebook: Facebook Graph API is widely used to integrate Facebook features into third-party applications. Developers can access user profiles, post updates, and perform social interactions through the API.
- Twitter: The Twitter API allows developers to retrieve tweets, post tweets, and access user data. It's used to embed Twitter feeds and share content on websites and applications.
- Instagram: The Instagram Graph API offers functionalities for displaying images, interacting with user posts, and authenticating users through their Instagram accounts. This enables seamless integration of Instagram content into apps.

2. Payment Gateways:

- PayPal: PayPal provides RESTful APIs that enable e-commerce platforms to accept payments, handle refunds, and manage transactions securely.
- Stripe: Stripe's API allows businesses to process payments, manage subscription billing, and handle various financial transactions, making it popular among startups and online businesses.
- Square: Square's APIs provide payment processing and point-of-sale solutions, allowing developers to build custom checkout experiences and integrate card payment functionality.

3. Maps and Geolocation:

- Google Maps: The Google Maps JavaScript API lets developers embed interactive maps, geolocation, and directions into websites and mobile apps. It's commonly used for location-based services.
- Mapbox: Mapbox offers APIs for creating custom maps, geospatial data visualization, and location-based applications, appealing to developers who need advanced customization and styling options.

4. Weather Information:

- OpenWeatherMap: OpenWeatherMap's API provides real-time and historical weather data. Developers use it to display current weather conditions, forecasts, and climate information in their applications.
- Weather.com: The Weather Channel API offers access to weather forecasts, radar data, and severe weather alerts, making it a valuable resource for weather-related applications and services.

5. Financial Data:

- Alpha Vantage: Alpha Vantage's API provides financial data, including stock market information, exchange rates, and cryptocurrency data. It's used by developers creating investment and trading apps.
- Quandl: Quandl offers an API for accessing financial and economic data, making it a valuable resource for data-driven financial research and analysis.

6. Cloud Services:

- AWS (Amazon Web Services): AWS offers a comprehensive set of APIs for managing cloud infrastructure, computing, storage, and other cloud services. Developers can automate the deployment and scaling of applications using AWS services.

- Google Cloud: Google Cloud's APIs enable developers to interact with a wide range of cloud-based tools and services, from machine learning and data storage to analytics and cloud computing.

7. IoT and Home Automation:

- Smart home devices, like smart thermostats (e.g., Nest), smart lights (e.g., Philips Hue), and voice assistants (e.g., Amazon Alexa), provide APIs for developers to create custom automation routines, control devices remotely, and integrate them with other IoT solutions.

These are just a few examples of how APIs are integral to diverse industries, enabling developers to enhance the functionality of their applications by tapping into external data and services seamlessly. The versatility and accessibility of APIs make them a cornerstone of modern software development.

Advantages of APIs:

1. Interoperability:

- Cross-Platform Integration: One of the primary advantages of APIs is their ability to bridge the gap between different technologies, platforms, and systems. For example, a mobile app running on iOS can communicate with a server running on Linux, thanks to APIs. This interoperability is crucial in today's multi-device and multi-platform world.

2. Efficiency:

- Reusable Code: APIs provide a way to reuse code and functionality. Instead of building every component from scratch, developers can tap into existing APIs to access common functionalities. This not only saves development time but also reduces the likelihood of errors, as the code within the API is typically well-tested and optimized.

- Rapid Development: By using APIs, developers can expedite the development process. For example, integrating a payment gateway into an e-commerce website is much faster using a payment API than writing the entire payment processing system from scratch.

3. Scalability:

- Distributed Architecture: APIs facilitate scalable solutions by allowing services to be distributed across different servers or systems. When traffic increases, additional servers can be added to handle the load. Cloud-based APIs are particularly adept at this, as cloud providers can allocate resources dynamically to accommodate growing demand.
- Load Balancing: Load balancing is simplified with APIs. Requests can be distributed across multiple servers, ensuring that no single server is overwhelmed with traffic, thereby enhancing the system's overall scalability and reliability.

4. Innovation:

- Ecosystem Growth: APIs foster innovation by making functionalities and data accessible to a broader developer community. When companies provide open APIs, third-party developers can build upon their platforms, creating new and diverse applications and services. This leads to an ecosystem of innovation around the core service or product.
- Customization: Developers can use APIs to customize and extend the functionality of existing applications. For instance, social media platforms often offer APIs to allow third-party developers to create apps that enhance the social experience.

5. Improved User Experience:

- Feature Expansion: APIs enable developers to integrate third-party services and features seamlessly into their applications. For example, integrating a mapping API into

a ride-sharing app enhances the user experience by providing accurate location information.

- More Choices: By leveraging APIs, developers can offer users more choices. For example, a food delivery app can offer a variety of payment methods by integrating multiple payment gateway APIs, catering to a wider range of customer preferences.

In conclusion, APIs are instrumental in modern software development due to their ability to enhance interoperability, efficiency, scalability, and innovation. They empower developers to create feature-rich applications that provide better user experiences, while also fostering a thriving ecosystem of innovation and collaboration within the software development community.

Disadvantages of APIs:

1. Security Concerns:

- Data Exposure: APIs, if not properly secured, can become entry points for cyberattacks. They might inadvertently expose sensitive data, such as user information or financial details, to malicious actors.
- Authorization and Authentication: Securing APIs involves implementing robust authorization and authentication mechanisms. Inadequate security measures can lead to unauthorized access, data breaches, and other security vulnerabilities.
- API Vulnerabilities: Vulnerabilities such as SQL injection, Cross-Site Scripting (XSS), or Cross-Site Request Forgery (CSRF) can be exploited through poorly designed or unprotected APIs.

2. Reliability Issues:

- Downtime: If an API provider experiences downtime or service interruptions, it can have a cascading effect on the applications or services relying on that API. The

reliability of third-party APIs can directly impact the uptime and functionality of dependent applications.

- Changes in API Versions: API providers may update or change their APIs over time. This can lead to compatibility issues for applications that were built based on a previous API version, necessitating updates and modifications to maintain functionality.

3. Overhead:

- Network Traffic: Overusing APIs or making excessive API calls can result in increased network traffic. This can lead to performance bottlenecks and increased latency in applications.
- Rate Limiting: To prevent abuse, many API providers enforce rate limiting, restricting the number of requests a client can make within a specific time frame. Exceeding these limits can lead to throttling or even temporary bans.

4. Data Privacy:

- Uninformed Data Sharing: Users may be unaware of the extent to which their data is shared through APIs. This lack of transparency can raise concerns about data privacy and lead to regulatory or public backlash.
- Data Mishandling: Improper handling of user data through APIs can result in privacy breaches and violations of data protection laws, which can have legal and reputational consequences.

5. Maintenance:

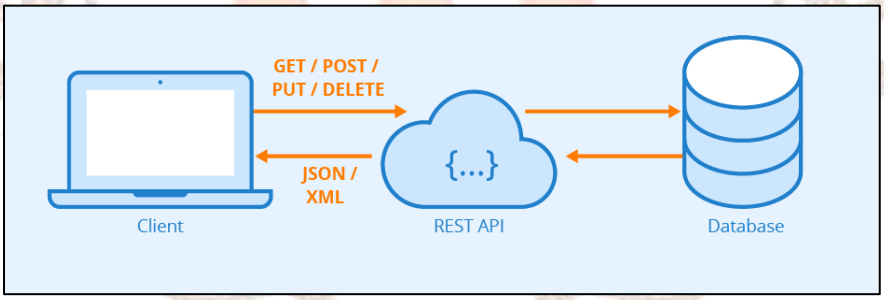
- Version Compatibility: As technology evolves, APIs may require updates to remain compatible with the latest standards and security practices. This ongoing maintenance can be resource-intensive and necessitate changes to applications that depend on those APIs.

- Deprecation: API providers may deprecate older versions of their APIs, which can necessitate significant changes to applications that rely on them. Developers must stay vigilant about deprecation notices and update their systems accordingly.

In conclusion, while APIs offer numerous advantages, they also come with potential drawbacks related to security, reliability, performance, data privacy, and ongoing maintenance. Developers and organizations must carefully consider these disadvantages and implement best practices for API design, security, and management to mitigate these risks and ensure that APIs are used safely and effectively.

3. UNDERSTANDING REST APIs

Representational **S**tate **T**ransfer (REST) is an architectural style that defines a set of constraints to be used for creating web services. **REST API** is a way of accessing web services in a simple and flexible way without having any processing.



REST technology is generally preferred to the more robust Simple Object Access Protocol (SOAP) technology because REST uses less bandwidth, simple and flexible making it more suitable for internet usage. It's used to fetch or give some information from a web service. All communication done via REST API uses only HTTP request.

Working: A request is sent from client to server in the form of a web URL as HTTP GET or POST or PUT or DELETE request. After that, a response comes back from the server in the form of a resource which can be anything like HTML, XML, Image, or JSON. But now JSON is the most popular format being used in Web Services.

In **HTTP** there are five methods that are commonly used in a REST-based Architecture i.e., POST, GET, PUT, PATCH, and DELETE. These correspond to create, read, update, and delete

(or CRUD) operations respectively. There are other methods which are less frequently used like OPTIONS and HEAD.

- **GET:** The HTTP GET method is used to **read** (or retrieve) a representation of a resource. In the safe path, GET returns a representation in XML or JSON and an HTTP response code of 200 (OK). In an error case, it most often returns a 404 (NOT FOUND) or 400 (BAD REQUEST).
- **POST:** The POST verb is most often utilized to **create** new resources. In particular, it's used to create subordinate resources. That is, subordinate to some other (e.g. parent) resource. On successful creation, return HTTP status 201, returning a Location header with a link to the newly-created resource with the 201 HTTP status.
- **PUT:** It is used for **updating** the capabilities. However, PUT can also be used to **create** a resource in the case where the resource ID is chosen by the client instead of by the server. In other words, if the PUT is to a URI that contains the value of a non-existent resource ID. On successful update, return 200 (or 204 if not returning any content in the body) from a PUT. If using PUT for create, return HTTP status 201 on successful creation. PUT is not safe operation but it's idempotent.
- **PATCH:** It is used to **modify** capabilities. The PATCH request only needs to contain the changes to the resource, not the complete resource. This resembles PUT, but the body contains a set of instructions describing how a resource currently residing on the server should be modified to produce a new version. This means that the PATCH body should not just be a modified part of the resource, but in some kind of patch language like JSON Patch or XML Patch. PATCH is neither safe nor idempotent.
- **DELETE:** It is used to **delete** a resource identified by a URI. On successful deletion, return HTTP status 200 (OK) along with a response body.

Idempotence: An idempotent HTTP method is a HTTP method that can be called many times without different outcomes. It would not matter if the method is called only once, or ten times over. The result should be the same. Again, this only applies to the result, not the resource itself.

Request and Response

Now we will see how request and response work for different **HTTP** methods. Let's assume we have an **API(**https://www.example.org/api/students**)** for all students data of xxx

- **GET:** Request for all Students.

Request
GET:/api/students

- **POST:** Request for Posting/Creating/Inserting Data

Request
POST:/api/students {"name":"Raj"}

- **PUT or PATCH:** Request for Updating Data at id=1

Request
PUT or PATCH:/api/students/1 {"name":"Raj"}

- **DELETE:** Request for Deleting Data of id=1

Request
DELETE:/api/students/1

RESTful web services are very popular because they are light weight, highly scalable and maintainable and are very commonly used to create APIs for web-based applications.

4. FETCHING DATA FROM APIS USING JAVASCRIPT

Fetching data from APIs using JavaScript is a common task in web development, enabling you to retrieve data from remote servers and integrate it into your web applications. This process is vital for creating dynamic and interactive websites or web apps. Here's a detailed explanation of how to fetch data from APIs using JavaScript:

1. Understanding APIs:

- An API (Application Programming Interface) defines a set of rules and protocols for how different software components should interact. In the context of web development, APIs allow your web application to communicate with external services, such as data sources, social media platforms, or other web applications.

2. Choosing an API:

- You first need to identify the API that provides the data you want to fetch. Many services and platforms offer APIs with documentation that explains how to access their data and what data is available.

3. Making HTTP Requests:

- To fetch data from an API, you use HTTP requests. JavaScript provides several methods for making HTTP requests, with two of the most common being:
- Using the `XMLHttpRequest` Object: This traditional approach allows you to create and send HTTP requests. However, it can be complex and is less commonly used today.
- Using the `fetch()` API: The `fetch()` function is a modern and more user-friendly method for making HTTP requests. It returns a Promise, making it easier to work with asynchronous operations.

The syntax for the fetch() method is

fetch(URL, options)

The fetch() method accepts two arguments:

- URL: The URL to access
    - options: An object which consists of additional properties (methods, headers) sent to the server.

Without options, we will only be downloading contents of the URL

Fetching for resources

To fetch for some resource, you will call the fetch() method, and pass it one argument : aURL (which indicates the path to the resource you want to fetch).

const promise = fetch(url)

The fetch() method returns a Promise, which can either be resolved or rejected.

- The promise resolves into a Response object. The Response object has a number of useful properties and methods to explore what we got from the request.
- The promise rejects if the fetch() method was unable to make the HTTP-request. Rejections can be due to network problems, or if no such URL exits.

We chain the fetch() method with a then() and catch() method to enable us to use the response.

- The then() method is basically used to handle asynchronous tasks (tasks that are time-consuming), such as fetching data from an API. When we get a response from the fetch() method, we will handle the response in the then().
- Usually, we get a Response object from the fetch request. This object contains useful properties and methods.
- If there is an error in fetching the request, for instance due to network problems or invalid URL, the catch() method will handle this error.

The complete syntax for fetching information from a server is as below:

```
fetch(url)
  .then(response => {
    // handle the response object
  })
  .catch(error => {
    // handle the error
  });
```

Reading the Response object

The Response object contains further data that needs to be converted into the required format in order to work with it.

There are several methods you can use to read the body of the response object, one such method is the json() method.

- The json() method returns another Promise that decides the complete data of the fetched resource.
- We then access the data in another .then() method

Example1: Fetching and reading data from an API

In the code below, we will fetch a dummy phone product from the dummyjson.com server, and then log the fetched data.

```
fetch('https://dummyjson.com/products/1')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Steps for fetching and reading data from an API

- Pass the fetch() method the required url to get the data from. In this instance, the URL will be https://dummyjson.com/products/1
- Chain the fetch() method with the then() method
- If all goes well, in the then() method, we will receive a Response object from the server
- Extract the JSON body content from the Response object using the json() method.
- The json() method will return a secondPromise. This Promise will resolve to contain the **fetched data**
    - Pass this data to another then() method.
- In the then()method, we can now print the fetched data to the console or assign it to a variable.
- If there is an error in fetching the data, the catch() method will capture the error, and we can log that error

Example 2: Fetching data from fake store API

In this instance, we will fetch a single ecommerce product from the Fake Store API. The fake Store API is a free online REST API we will be using to fetch pseudo-real data for our ecommerce website.

- Call the fetch() method and pass the url or API endpoint as the argument.
- The API endpoint or the url for this instance is : 'https://fakestoreapi.com/products/1'. It provides access to a single product

  fetch('https://fakestoreapi.com/products/1')
- Calling thisfetch() method returns a special JavaScript object called Promise. The Promise produces a value (Response object) after an asynchronous operation successfully completes, or an error if unsuccessful.
- Chain the fetch() method with athen() method to investigate the Response object we got from the API call.
- The Response object is assigned to a response variable. (You can give the Response object any variable name of choice, res or response is the convention)
- Use the json() method to view the complete content of the fetched resource (For instance res.json() or response.json() depending on how you named your variable)

  fetch('https://fakestoreapi.com/products/1').then(response => response.json())
- Finally, access the fetched data by chaining another .then() method

  fetch('https://fakestoreapi.com/products/1')

```
.then(
// view the  complete content of the response
response => response.json())
.then(
//access the actual data
data => console.log(data))
```

The output will be an object containing some dummy phone data- title, price, description etc.

{
  "id": 1,
  "title": "Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops",
  "price": 109.95,
  "description": "Your perfect pack for everyday use and walks in the forest. Stash your laptop (up to 15 inches) in the padded sleeve, your everyday",
  "category": "men's clothing",
  "image": "https://fakestoreapi.com/img/81fPKd-2AYL._AC_SL1500_.jpg",
  "rating": {
    "rate": 3.9,
    "count": 120
  }
}

- Instead of logging the output, you can assign this value to declared variable, and access the relevant data using the appropriate key:value pair.

Handling the status code of the response

In the Response object, you can use the status and statusText properties to access the status code and status text, respectively.

An HTTP status code is a server response to a browser's request. When you visit a website, your browser sends a request to the site's server, and the server then responds to the browser's request with a three-digit code: the HTTP status code.

In the code below, we read the status code and status text from the server's response:

```
fetch('https://dummyjson.com/products/1')
  .then(response => {
```

```
console.log(response.status) // check the status code of the response object
console.log(response.statusText) // an interpretation of the code
return response.json() // read the actual data from body of the response object
        })
 .then(data => console.log(data))
 .catch(err => console.error(err));
```

- A successful request will give a status code of 200 and a status text of OK
- If the requested URL throws a server error, the status code will be 500.
- If the requested resource does not exist, the status code will be 400 and status text will be Not found

Summary

- The Fetch API allows you to reqeust for a resource from a web server
- The Fetch API provides access to the fetch() method that returns a promise which resolves into a Response object
- Use the json() method to fetch the actual data
- Use the status and statusText properties of theResponse object to investigate the status of the response
- Use the catch() method to handle a failed request, for instance network problemsn

5. SUMMARY

In this introductory chapter, we embarked on a journey to understand the fundamental concepts of APIs, particularly focusing on the widely adopted REST (Representational State Transfer) APIs. We began by addressing the question, "What is an API?" and unraveling the role of APIs in modern web development. APIs were revealed as the essential bridges that facilitate seamless communication between diverse software systems.

Next, we delved into the intricate world of REST APIs, where we explored REST's core principles and architectural style. REST was unveiled as a set of constraints that guide the design of networked applications, emphasizing resource-oriented communication, statelessness, and the use of standard HTTP methods. This understanding laid the foundation for comprehending how REST APIs function and why they have become the industry standard for web service design.

The chapter concluded by delving into the practical aspects of data retrieval from APIs using JavaScript. We examined the various methods, libraries, and best practices for initiating HTTP requests to remote servers, extracting and processing data, and integrating this data into web applications. The discussion highlighted the importance of error handling, asynchronous programming, and API authentication.

By the end of this chapter, readers gained a comprehensive understanding of APIs, especially REST, and acquired the knowledge and tools necessary to fetch and manipulate data from APIs using JavaScript. This knowledge sets the stage for the exciting exploration of web development and data integration in the chapters to come.

6. SELF-ASSESSMENT QUESTIONS

1. What does the acronym "API" stand for?
2. Define the term "REST" in the context of web services.
3. What is the primary role of APIs in software development?
4. How does a RESTful API handle communication between clients and servers?
5. Which HTTP method is typically used to retrieve data from an API?
6. Name a commonly used JavaScript method for making HTTP requests to APIs.
7. Explain the importance of being stateless in RESTful communication.
8. How can HATEOAS enhance the flexibility of an API?
9. What are some benefits of using Promises when fetching data from APIs in JavaScript?

INSPIRED BY LIFE

7. TERMINAL QUESTIONS

1. Explain the concept of an API in software development. How do APIs facilitate communication between different software components and systems? Provide real-world examples to illustrate their significance.
2. Dive deep into the principles of REST architecture. Discuss the key constraints of RESTful APIs and how they influence the design and functionality of web services. Give examples of how these constraints are applied in practice.
3. Describe the process of making HTTP requests to a RESTful API using JavaScript. Walk through the steps involved, from initiating a request to handling the response. Highlight the role of asynchronous programming and error handling in this context.
4. Elaborate on the importance of resource-oriented communication in REST. How do resources and URIs play a crucial role in RESTful APIs? Provide examples of how resources are identified and manipulated using HTTP methods.
5. Discuss the role of authentication in securing API access. Explain the common methods of API authentication, such as API keys, tokens, and OAuth. Include best practices for implementing authentication in JavaScript when fetching data from APIs.
6. Consider the challenges and potential solutions related to cross-origin requests when fetching data from APIs located on different domains. Explain the concept of CORS (Cross-Origin Resource Sharing) and how it can impact API access in web development. Provide examples of how CORS can be configured and managed.

8. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. "API" stands for "Application Programming Interface."
2. In the context of web services, "REST" stands for "Representational State Transfer." It is an architectural style for designing networked applications.
3. The primary role of APIs in software development is to define how different software components or systems should interact with each other.
4. A RESTful API handles communication between clients and servers by using HTTP methods (e.g., GET, POST, PUT, DELETE) to perform operations on resources.
5. The HTTP method typically used to retrieve data from an API is "GET."

6. "fetch()" is a commonly used JavaScript method for making HTTP requests to APIs.
7. Being stateless in RESTful communication means that each request from a client to the server must contain all the information needed to understand and process the request, and there is no session state stored on the server between requests.
8. HATEOAS (Hypermedia as the Engine of Application State) enhances the flexibility of an API by allowing clients to dynamically discover and navigate through the API by following links provided in responses.
9. Promises are beneficial in JavaScript for handling asynchronous operations when fetching data from APIs. They provide a cleaner and more organized way to manage asynchronous code.
10. CORS (Cross-Origin Resource Sharing) comes into play when fetching data from APIs located on different domains. It is a security feature that determines whether web browsers should allow web pages to make requests to a different domain.

9. ANSWERS TO TERMINAL QUESTIONS

1. Refer Section 3
2. Refer Section 4
3. Refer Section 5
4. Refer Section 3
5. Refer Section 4
6. Refer Section 5